

Đáp Án Bài Tập Cuối Chương

.env

```
VITE_API_BASE_URL=http://localhost:5000/api
```

src/api/axiosClient.js

```
import axios from 'axios';

const axiosClient = axios.create({
  baseURL: process.env.VITE_API_BASE_URL,
  headers: {
    'Content-Type': 'application/json',
  },
});

export default axiosClient;
```

src/redux/store.js

```
import { configureStore } from '@reduxjs/toolkit';
import bookReducer from './slices/bookSlice';
import memberReducer from './slices/memberSlice';

export const store = configureStore({
  reducer: {
    books: bookReducer,
    members: memberReducer,
  },
});
```

src/redux/slices/bookSlice.js

```
import { createSlice, createAsyncThunk } from '@reduxjs/toolkit';
import axiosClient from '../api/axiosClient';

export const fetchBooks = createAsyncThunk('books/fetchBooks', async () => {
  const response = await axiosClient.get('/books');
  return response.data;
});

export const fetchBookById = createAsyncThunk('books/fetchBookById', async (bookId) => {
  const response = await axiosClient.get(`/books/${bookId}`);
  return response.data;
});

export const addBook = createAsyncThunk('books/addBook', async (title) => {
  const response = await axiosClient.post('/books', { title });
  return response.data;
});

export const toggleBorrowed = createAsyncThunk('books/toggleBorrowed', async ({ bookId, memberId }, { getState }) => {
  const book = getState().books.books.find(b => b.id === bookId);
  const updatedBook = { borrowed: !book.borrowed, borrowedBy: book.borrowed ? memberId : null };
  const response = await axiosClient.patch(`/books/${bookId}`, updatedBook);
  return response.data;
});
```

```

export const removeBook = createAsyncThunk('books/removeBook', async (bookId) => {
  await axiosClient.delete(`/books/${bookId}`);
  return bookId;
});

const bookSlice = createSlice({
  name: 'books',
  initialState: {
    books: [],
    currentBook: null,
    filter: 'all',
    loading: false,
    error: null,
  },
  reducers: {
    setFilter: (state, action) => {
      state.filter = action.payload;
    },
  },
  extraReducers: (builder) => {
    builder
      .addCase(fetchBooks.pending, (state) => {
        state.loading = true;
        state.error = null;
      })
      .addCase(fetchBooks.fulfilled, (state, action) => {
        state.loading = false;
        state.books = action.payload;
      })
      .addCase(fetchBooks.rejected, (state, action) => {
        state.loading = false;
        state.error = action.error.message;
      })
      .addCase(fetchBookById.pending, (state) => {
        state.loading = true;
        state.error = null;
      })
      .addCase(fetchBookById.fulfilled, (state, action) => {
        state.loading = false;
        state.currentBook = action.payload;
      })
      .addCase(fetchBookById.rejected, (state, action) => {
        state.loading = false;
        state.error = action.error.message;
      })
      .addCase(addBook.pending, (state) => {
        state.loading = true;
      })
      .addCase(addBook.fulfilled, (state, action) => {
        state.loading = false;
        state.books.push(action.payload);
      })
      .addCase(addBook.rejected, (state, action) => {
        state.loading = false;
        state.error = action.error.message;
      })
      .addCase(toggleBorrowed.fulfilled, (state, action) => {
        const index = state.books.findIndex(b => b.id === action.payload.id);
        if (index !== -1) state.books[index] = action.payload;
      });
  }
});

```

```

    })
    .addCase(removeBook.fulfilled, (state, action) => {
      state.books = state.books.filter(b => b.id !== action.payload);
    });
  },
});

```

```

export const { setFilter } = bookSlice.actions;
export default bookSlice.reducer;

```

src/redux/slices/memberSlice.js

```

import { createSlice, createAsyncThunk } from '@reduxjs/toolkit';
import axiosClient from '../api/axiosClient';

export const fetchMembers = createAsyncThunk('members/fetchMembers', async () => {
  const response = await axiosClient.get('/members');
  return response.data;
});

export const fetchMemberById = createAsyncThunk('members/fetchMemberById', async (memberId) => {
  const response = await axiosClient.get(`/members/${memberId}`);
  return response.data;
});

export const addMember = createAsyncThunk('members/addMember', async (name) => {
  const response = await axiosClient.post('/members', { name });
  return response.data;
});

export const removeMember = createAsyncThunk('members/removeMember', async (memberId) => {
  await axiosClient.delete(`/members/${memberId}`);
  return memberId;
});

const memberSlice = createSlice({
  name: 'members',
  initialState: {
    members: [],
    currentMember: null,
    loading: false,
    error: null,
  },
  reducers: {},
  extraReducers: (builder) => {
    builder
      .addCase(fetchMembers.pending, (state) => {
        state.loading = true;
        state.error = null;
      })
      .addCase(fetchMembers.fulfilled, (state, action) => {
        state.loading = false;
        state.members = action.payload;
      })
      .addCase(fetchMembers.rejected, (state, action) => {
        state.loading = false;
        state.error = action.error.message;
      })
      .addCase(fetchMemberById.pending, (state) => {
        state.loading = true;

```

```

    state.error = null;
  })
  .addCase(fetchMemberById.fulfilled, (state, action) => {
    state.loading = false;
    state.currentMember = action.payload;
  })
  .addCase(fetchMemberById.rejected, (state, action) => {
    state.loading = false;
    state.error = action.error.message;
  })
  .addCase(addMember.pending, (state) => {
    state.loading = true;
  })
  .addCase(addMember.fulfilled, (state, action) => {
    state.loading = false;
    state.members.push(action.payload);
  })
  .addCase(addMember.rejected, (state, action) => {
    state.loading = false;
    state.error = action.error.message;
  })
  .addCase(removeMember.fulfilled, (state, action) => {
    state.members = state.members.filter(m => m.id !== action.payload);
  })
  .addCase(removeMember.rejected, (state, action) => {
    state.error = action.error.message;
  });
},
});

export default memberSlice.reducer;

```

src/App.jsx

```

import { Provider } from 'react-redux';
import { BrowserRouter, Routes, Route } from 'react-router-dom';
import { store } from './redux/store';
import NavBar from './components/NavBar/NavBar';
import Home from './screens/Home/Home';
import BookDetail from './screens/BookDetail/BookDetail';
import MemberDetail from './screens/MemberDetail/MemberDetail';
import NotFound from './screens/NotFound/NotFound';

function App() {
  return (
    <Provider store={store}>
      <BrowserRouter>
        <NavBar />
        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/book/:id" element={<BookDetail />} />
          <Route path="/member/:id" element={<MemberDetail />} />
          <Route path="*" element={<NotFound />} />
        </Routes>
      </BrowserRouter>
    </Provider>
  );
}

```

```
export default App;
```

src/screens/Home/Home.jsx

```
import { useState, useEffect } from 'react';
import { useDispatch, useSelector } from 'react-redux';
import { fetchBooks, addBook } from '../../redux/slices/bookSlice';
import { fetchMembers, addMember } from '../../redux/slices/memberSlice';
import BookList from '../../components/BookList/BookList';
import MemberList from '../../components/MemberList/MemberList';
import FilterButtons from '../../components/FilterButtons/FilterButtons';

function Home() {
  const [bookTitle, setBookTitle] = useState('');
  const [memberName, setMemberName] = useState('');
  const dispatch = useDispatch();
  const bookLoading = useSelector(state => state.books.loading);
  const memberLoading = useSelector(state => state.members.loading);
  const error = useSelector(state => state.books.error || state.members.error);

  useEffect(() => {
    dispatch(fetchBooks());
    dispatch(fetchMembers());
  }, [dispatch]);

  const handleAddBook = () => {
    if (bookTitle.trim()) {
      dispatch(addBook(bookTitle));
      setBookTitle('');
    }
  };

  const handleAddMember = () => {
    if (memberName.trim()) {
      dispatch(addMember(memberName));
      setMemberName('');
    }
  };

  if (bookLoading || memberLoading) return <p>Loading...</p>;
  if (error) return <p>Error: {error}</p>;

  return (
    <div>
      <h1>Library Management</h1>
      <div>
        <input value={bookTitle} onChange={(e) => setBookTitle(e.target.value)} placeholder="Enter book title" />
        <button onClick={handleAddBook}>Add Book</button>
      </div>
      <div>
        <input value={memberName} onChange={(e) => setMemberName(e.target.value)} placeholder="Enter member name" />
        <button onClick={handleAddMember}>Add Member</button>
      </div>
      <FilterButtons />
      <BookList />
      <MemberList />
    </div>
  );
}
```

```
);  
}  
  
export default Home;
```

src/components/BookList/BookList.jsx

```
import { useSelector } from 'react-redux';  
import BookItem from '../BookItem/BookItem';  
  
function BookList() {  
  const { books, filter } = useSelector(state => state.books);  
  
  const filteredBooks = books.filter(book => {  
    if (filter === 'borrowed') return book.borrowed;  
    if (filter === 'available') return !book.borrowed;  
    return true;  
  });  
  
  return (  
    <div className="book-list">  
      <h2>Books</h2>  
      {filteredBooks.map(book => (  
        <BookItem key={book.id} book={book} />  
      ))}  
    </div>  
  );  
}
```

export default BookList;

src/components/BookItem/BookItem.jsx

```
import { useDispatch, useSelector } from 'react-redux';  
import { Link } from 'react-router-dom';  
import { removeBook, toggleBorrowed } from '../../redux/slices/bookSlice';  
  
function BookItem({ book }) {  
  const dispatch = useDispatch();  
  const members = useSelector(state => state.members.members);  
  
  const handleBorrowChange = (e) => {  
    const memberId = e.target.value === 'available' ? null : e.target.value;  
    dispatch(toggleBorrowed({ bookId: book.id, memberId }));  
  };  
  
  return (  
    <div className="book-item">  
      <Link to={`/${book.id}`}>{book.title}</Link>  
      <select value={book.borrowedBy || 'available'} onChange={handleBorrowChange}>  
        <option value="available">Available</option>  
        {members.map(member => (  
          <option key={member.id} value={member.id}>{member.name}</option>  
        ))}  
      </select>  
      <button onClick={() => dispatch(removeBook(book.id))}>Delete</button>  
    </div>  
  );  
}
```



```
export default BookItem;
```

src/components/MemberList/MemberList.jsx

```
import { useSelector } from 'react-redux';
import MemberItem from '../MemberItem/MemberItem';

function MemberList() {
  const members = useSelector(state => state.members.members);

  return (
    <div className="member-list">
      <h2>Members</h2>
      {members.map(member => (
        <MemberItem key={member.id} member={member} />
      ))}
    </div>
  );
}

export default MemberList;
```

src/components/MemberItem/MemberItem.jsx

```
import { useDispatch } from 'react-redux';
import { Link } from 'react-router-dom';
import { removeMember } from '../../redux/slices/memberSlice';

function MemberItem({ member }) {
  const dispatch = useDispatch();

  const handleDelete = () => {
    dispatch(removeMember(member.id));
  };

  return (
    <div className="member-item">
      <Link to={`/${member.id}`}>{member.name}</Link>
      <button onClick={handleDelete}>Delete</button>
    </div>
  );
}

export default MemberItem;
```

src/components/FilterButtons/FilterButtons.jsx

```
import { useDispatch } from 'react-redux';
import { setFilter } from '../../redux/slices/bookSlice';

function FilterButtons() {
  const dispatch = useDispatch();

  return (
    <div className="filter-buttons">
      <button onClick={() => dispatch(setFilter('all'))}>All</button>
      <button onClick={() => dispatch(setFilter('borrowed'))}>Borrowed</button>
      <button onClick={() => dispatch(setFilter('available'))}>Available</button>
    </div>
  );
}
```



```
);  
}  
  
export default FilterButtons;
```

src/screens/BookDetail/BookDetail.jsx

```
import { useEffect } from 'react';  
import { useParams, Link } from 'react-router-dom';  
import { useDispatch, useSelector } from 'react-redux';  
import { fetchBookById } from '../../redux/slices/bookSlice';  
  
function BookDetail() {  
  const { id } = useParams();  
  const dispatch = useDispatch();  
  const book = useSelector(state => state.books.currentBook);  
  const members = useSelector(state => state.members.members);  
  const loading = useSelector(state => state.books.loading);  
  const error = useSelector(state => state.books.error);  
  
  useEffect(() => {  
    dispatch(fetchBookById(id));  
  }, [dispatch, id]);  
  
  if (loading) return <p>Loading...</p>;  
  if (error) {  
    return (  
      <div>  
        <p>{error}</p>  
        <Link to="/">Back to Home</Link>  
      </div>  
    );  
  }  
  if (!book || book.id !== id) return null;  
  
  const member = members.find(m => m.id === book.borrowedBy);  
  
  return (  
    <div>  
      <h1>{book.title}</h1>  
      <p>Status: {book.borrowed ? 'Borrowed' : 'Available'}</p>  
      {book.borrowed && member && (  
        <p>Borrowed by: <Link to={`/member/${member.id}`}>{member.name}</Link></p>  
      )}  
      <p>Added at: {new Date(book.addedAt).toLocaleString()}</p>  
      <Link to="/">Back</Link>  
    </div>  
  );  
}
```

export default BookDetail;

src/screens/MemberDetail/MemberDetail.jsx

```
import { useEffect } from 'react';  
import { useParams, Link } from 'react-router-dom';  
import { useDispatch, useSelector } from 'react-redux';  
import { fetchMemberById } from '../../redux/slices/memberSlice';  
  
function MemberDetail() {
```



```

const { id } = useParams();
const dispatch = useDispatch();
const member = useSelector(state => state.members.currentMember);
const books = useSelector(state => state.books.books);
const loading = useSelector(state => state.members.loading);
const error = useSelector(state => state.members.error);

useEffect(() => {
  dispatch(fetchMemberById(id));
}, [dispatch, id]);

if (loading) return <p>Loading...</p>;
if (error) {
  return (
    <div>
      <p>{error}</p>
      <Link to="/">Back to Home</Link>
    </div>
  );
}
if (!member || member.id !== id) return null;

const borrowedBooks = books.filter(b => b.borrowedBy === id);

return (
  <div>
    <h1>{member.name}</h1>
    <p>Joined at: {new Date(member.joinedAt).toLocaleString()}</p>
    <h3>Borrowed Books:</h3>
    {borrowedBooks.length > 0 ? (
      <ul>
        {borrowedBooks.map(book => (
          <li key={book.id}><Link to={`/book/${book.id}`}>{book.title}</Link></li>
        ))}
      </ul>
    ) : (
      <p>No books borrowed</p>
    )}
    <Link to="/">Back</Link>
  </div>
);
}

export default MemberDetail;

```

src/screens/NotFound/NotFound.jsx

```

import { Link } from 'react-router-dom';

function NotFound() {
  return (
    <div>
      <h1>404 - Page Not Found</h1>
      <Link to="/">Back to Home</Link>
    </div>
  );
}

export default NotFound;

```

src/components/NavBar/NavBar.jsx

```
import { Link } from 'react-router-dom';

function NavBar() {
  return (
    <nav className="navbar">
      <Link to="/">Library Manager</Link>
    </nav>
  );
}

export default NavBar;
```

src/main.jsx

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App';

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```